LR2 — Containerization and Web Application Deployment

Topic

Automated deployment and management of an ASP.NET Core web application using Azure VM and Docker.

Objective

Gain practical experience in deploying and managing an ASP.NET Core web application using Docker containerization and Azure cloud infrastructure provisioned with Terraform.

Steps Performed

1. ASP.NET Core Application

Created a **Workout Planner** web application using .NET 10 with:

- Home page with workout parameter form (age, weight, height, fitness goal, fitness level)
- Prediction API endpoint (`/api/prediction`) for ML integration
- Health check endpoint (`/health`) for monitoring
- MVC architecture with Controllers, Models, and Views

2. Dockerfile

Multi-stage Docker build:

```
# Stage 1: Build with SDK
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
# Stage 2: Runtime with ASP.NET runtime only
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
```

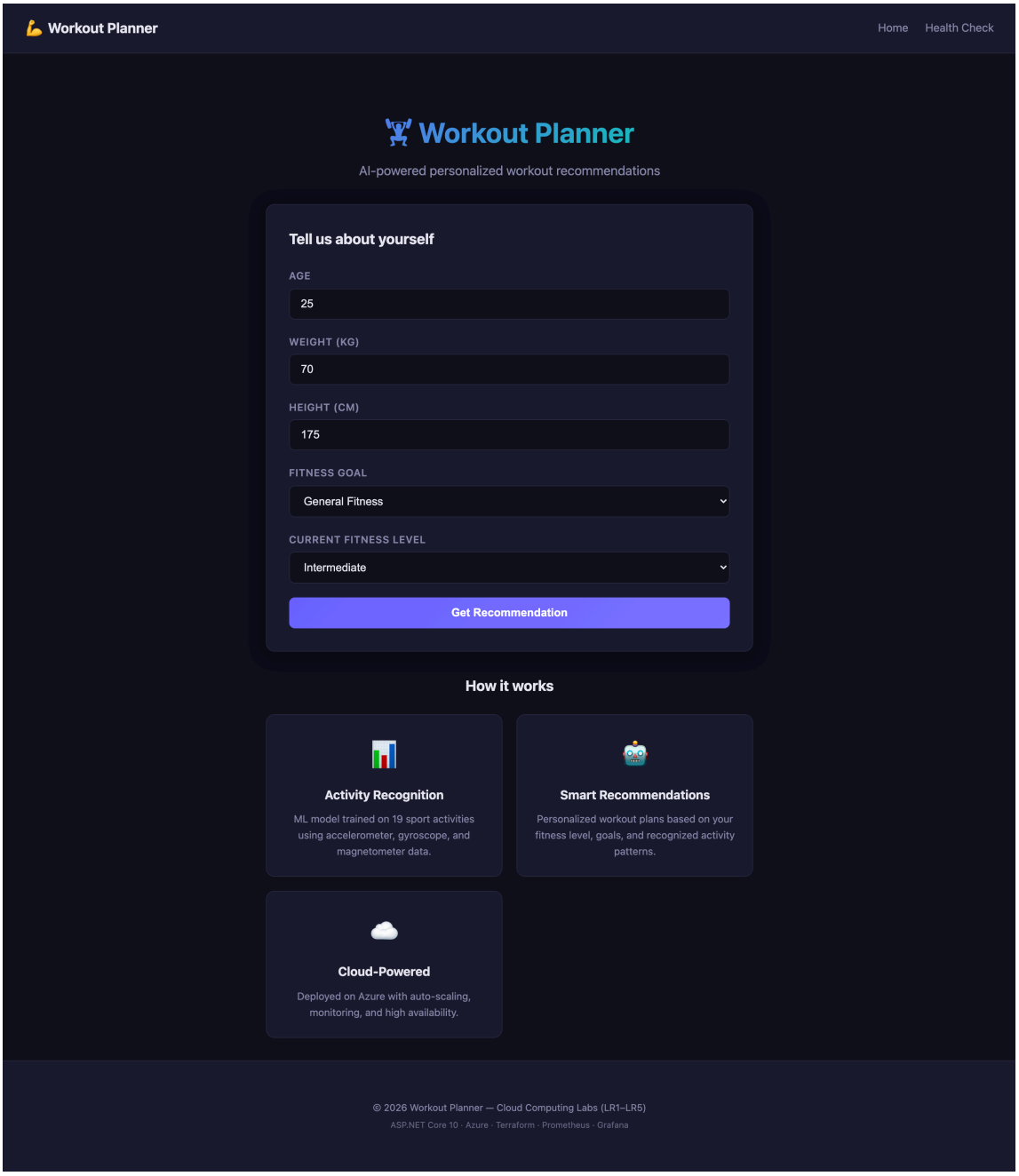
Key features:

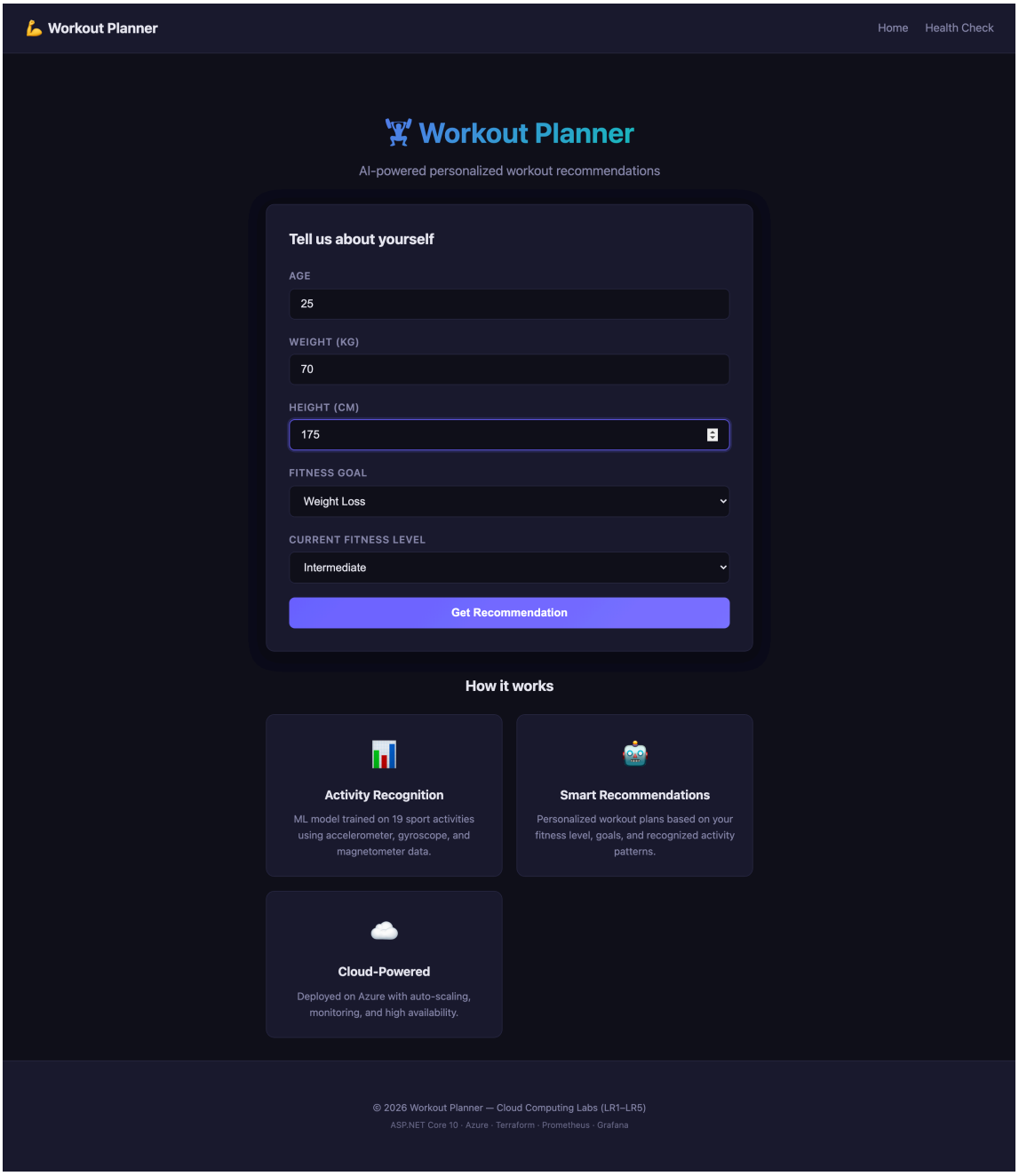
- Multi-stage build reduces final image size (~200MB vs ~2GB)
- Non-root user (`appuser`) for security
- Ports 80 (HTTP) and 443 (HTTPS) exposed

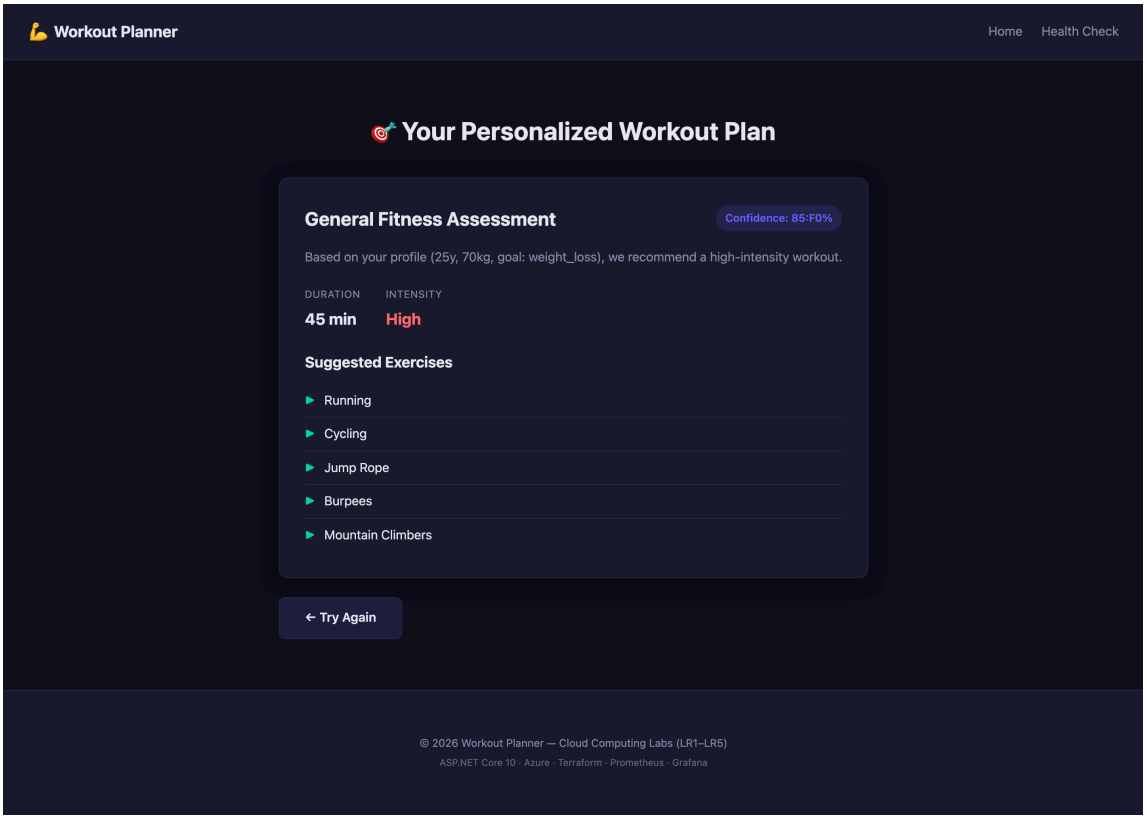
3. Local Docker Run

```
docker build -t workoutplanner:latest ./app/web/
docker run -d -p 8080:80 --name wp workoutplanner:latest
curl http://localhost:8080/health
```

Screenshots:




Workout Planner
Home
Health Check
Workout Planner
AI-powered personalized workout recommendations
Tell us about yourself
AGE
25
WEIGHT (KG)
70
HEIGHT (CM)
175
FITNESS GOAL
Weight Loss
CURRENT FITNESS LEVEL
Intermediate
Get Recommendation
How it works
Activity Recognition
ML model trained on 19 sport activities using accelerometer, gyroscope, and magnetometer data.
Smart Recommendations
Personalized workout plans based on your fitness level, goals, and recognized activity patterns.
Cloud-Powered
Deployed on Azure with auto-scaling, monitoring, and high availability.
© 2026 Workout Planner — Cloud Computing Labs (LR1–LR5)
ASP.NET Core 10 · Azure · Terraform · Prometheus · Grafana



Pretty-print

{"status":"healthy","timestamp":"2026-04-17T16:54:29.7861474Z","version":"1.0.0"}

4. Terraform Infrastructure

Core (`infra/core/`)

- `azurerm_resource_group` — single resource group for the entire project
- `azurerm_virtual_network` — VNet with `10.0.0.0/16` address space
- `azurerm_subnet` — app subnet (`10.0.1.0/24`) and k8s subnet (`10.0.2.0/24`)
- `azurerm_network_security_group` — rules for HTTP (80), HTTPS (443), SSH (22), App (8080)

Container Registry (`infra/registry/`)

- `azurerm_container_registry` — Basic SKU, admin enabled
- Login server: `acrworkoutplanner.azurecr.io`

VM Deployment (`infra/app-vm/`)

- `azurerm_linux_virtual_machine` — Ubuntu 22.04, Standard_B1s
- `azurerm_public_ip` — static public IP
- Cloud-init script installs Docker, pulls image from ACR, and runs the container

5. Container Registry

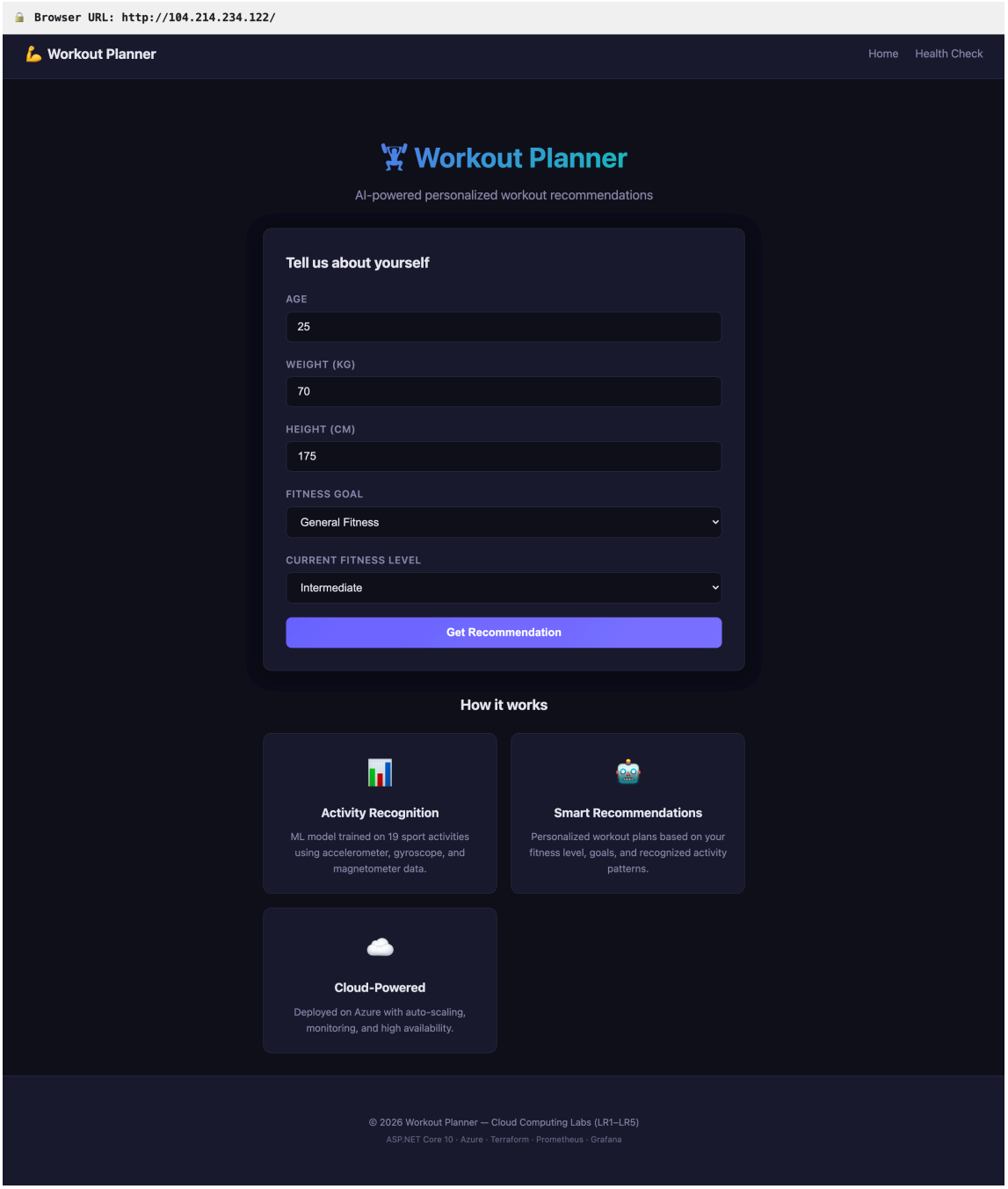
```
az acr login --name acrworkoutplanner
docker tag workoutplanner:latest acrworkoutplanner.azurecr.io/workoutplanner:latest
docker push acrworkoutplanner.azurecr.io/workoutplanner:latest
```

Screenshot: *ACR deployed as via script*

6. Deployed Application

- **URL**: `http://<VM_PUBLIC_IP>`
- **HTTPS**: Configured via NSG rules (port 443 open)

Screenshots:



Browser URL: http://104.214.234.122/

Workout Planner

Home Health Check

Workout Planner

AI-powered personalized workout recommendations

Tell us about yourself

AGE

30

WEIGHT (KG)

80

HEIGHT (CM)

180

FITNESS GOAL

Muscle Gain

CURRENT FITNESS LEVEL

Intermediate

Get Recommendation

How it works

Activity Recognition

ML model trained on 19 sport activities using accelerometer, gyroscope, and magnetometer data.

Smart Recommendations

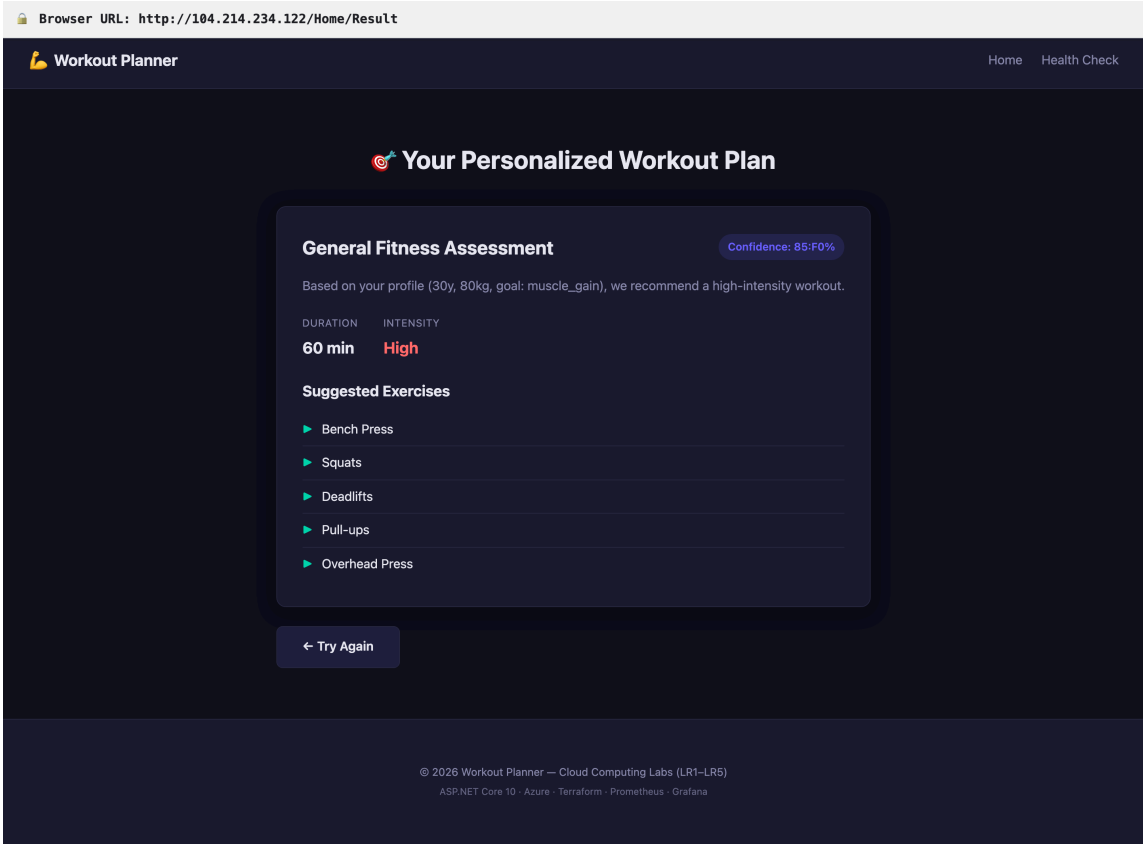
Personalized workout plans based on your fitness level, goals, and recognized activity patterns.

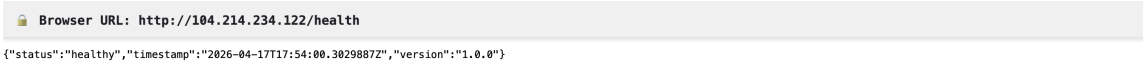
Cloud-Powered

Deployed on Azure with auto-scaling, monitoring, and high availability.

© 2026 Workout Planner — Cloud Computing Labs (LR1–LR5)

ASP.NET Core 10 · Azure · Terraform · Prometheus · Grafana





{"status":"healthy","timestamp":"2026-04-17T17:54:00.3029887Z","version":"1.0.0"}

7. IAM / RBAC

- VM uses SSH key authentication (password authentication disabled)

- ACR admin credentials used for initial setup
- NSG restricts inbound traffic to HTTP/HTTPS/SSH only

Port Mapping

Container Port	Host Port	Protocol
80	80	HTTP
443	443	HTTPS

Terraform Commands

```
cd infra/core && terraform init && terraform apply
cd ../registry && terraform init && terraform apply
# Build and push image
./scripts/build-and-push.sh
cd infra/app-vm && terraform init && terraform apply
```

Results

- ASP.NET Core 10 application successfully containerized
- Docker image pushed to Azure Container Registry
- Application deployed on Azure VM via Terraform
- HTTP/HTTPS access confirmed
- Health check endpoint operational

Conclusion

The lab demonstrated the full lifecycle of containerizing an ASP.NET Core web application and deploying it to Azure infrastructure managed by Terraform. The use of Infrastructure as Code ensures reproducibility and version control of all cloud resources. Docker multi-stage builds optimize the final image size, and cloud-init automates the VM setup process.